DESIGN AND ANALYSIS OF ALGORITHMS

Assignment No. 8

OPTIMIZING DELIVERY ROUTES
FOR A LOGISTICS COMPANY

Travelling Salesman Problem
Least Cost Branch and Bound

Name	Heet Oswal
PRN	124B1F141
Department	Information Technology
Assignment Topic	Travelling Salesman Problem
Algorithm	Least Cost Branch and Bound
Programming Language	C++

Pimpri Chinchwad College of Engineering, Pune

Department of Information Technology

1. Problem Statement

A leading logistics company, SwiftShip, is responsible for delivering packages to multiple cities. To minimize fuel costs and delivery time, the company needs to find the shortest possible route that allows a delivery truck to visit every city exactly once and return to the starting point.

The problem involves varying distances between cities, fuel consumption costs depending on road conditions, and time constraints for completing deliveries.

For N cities, a brute-force approach examines $(N-1)!$ possible routes, which becomes computationally expensive as the number of cities increases.

Therefore, a Branch and Bound algorithm is used to explore promising routes and eliminate partial solutions that cannot improve the best solution found so far.

Tasks:

1. Model the delivery network as a weighted graph.
2. Implement a Branch and Bound algorithm to find a minimum-cost delivery route.
3. Reduce unnecessary computations by pruning unpromising partial routes.
4. Determine the minimum route cost and the corresponding optimal route.

2. Course Objectives

1. To understand the basics of computational complexity of various algorithms.
2. To select appropriate algorithm design strategies to solve real-world problems.

3. Course Outcomes

1. Apply the Branch and Bound technique to solve problems.
2. Classify P, NP, NP-complete, and NP-hard problems.

4. Theory

4.1 Travelling Salesman Problem

The Travelling Salesman Problem (TSP) is a combinatorial optimization problem in which a salesman must visit every city exactly once and return to the starting city while minimizing the total travel cost.

The cities are represented as vertices, and the travel costs between cities are represented as weighted edges.

The objective is to find a minimum-cost Hamiltonian cycle.

4.2 Branch and Bound

Branch and Bound is an algorithmic technique used to solve optimization problems by systematically exploring partial solutions.

Each node in the search tree represents a partial route and contains the following information:

- **Partial path:** The cities visited so far.
- **Current cost:** The cost accumulated along the partial route.
- **Visited array:** Tracks the cities already visited.
- **Lower bound:** An estimate of the minimum possible cost of completing the route.

The algorithm expands promising nodes and eliminates partial routes whose lower bound cannot improve the best complete route found so far.

4.3 Least Cost Branch and Bound

In Least Cost Branch and Bound, nodes are selected for expansion in increasing order of their lower-bound estimates.

A priority queue is used to retrieve the node with the smallest bound first.

If a node's bound is greater than or equal to the best complete route cost, it is pruned because it cannot produce a better solution under the bound used.

This strategy can reduce the search space substantially compared with examining every possible permutation.

4.4 Lower-Bound Calculation in the Submitted Program

The submitted C++ implementation estimates the lower bound by adding the current path cost to the minimum outgoing edge cost for each unvisited city and the current endpoint.

For a node u, the program uses the following general form:

$$\text{Bound}(u) = \text{CurrentCost}(u) + \sum_{i \in U(u)} \min_{j \neq i} c_{ij}$$

where $U(u)$ represents the unvisited cities together with the current endpoint, as handled by the implementation.

The resulting estimate is used to prioritize nodes and prune search branches.

Note: This is the lower-bound method implemented in the submitted program. It is different from the reduced-cost matrix technique described in the reference manual.

5. Working of the Algorithm

1. Read the number of cities and the cost matrix.

2. Start the search from city 0.
3. Mark the starting city as visited and initialize the current route cost to zero.
4. Calculate the initial lower bound and insert the starting node into a minimum-priority queue.
5. Remove the node having the smallest bound.
6. If its bound is not better than the current best cost, discard the node.
7. If all cities have been visited, add the return cost to the starting city and update the best route if necessary.
8. Otherwise, generate child nodes for unvisited cities connected by a positive-cost edge.
9. Calculate the bound for every child and insert it into the priority queue only if it can improve the best solution.
10. Continue until the priority queue becomes empty.
11. Display the minimum route cost and the optimal route if a complete tour exists.

6. Pseudocode

```
procedure TSP_BranchAndBound(costMatrix):

    Initialize bestCost = infinity
    Initialize bestPath = empty

    Create a priority queue ordered by lower bound

    Create the root node:
        path = [0]
        mark city 0 as visited
        currentCost = 0
        bound = calculateBound(root)

    Insert root into the priority queue

    while priority queue is not empty:

        Remove the node with the smallest bound

        If node.bound >= bestCost:
            Continue

        If all cities have been visited:
            totalCost = currentCost
                        + cost[lastCity][0]

            If totalCost < bestCost:
                Update bestCost
                Store the complete route

            Continue
```

```
        For each unvisited city:
            If an edge exists from the current city:
                Create a child node
                Add the edge cost to currentCost
                Mark the city as visited
                Calculate the child's bound

                If child.bound < bestCost:
                    Insert child into the queue

    Return bestCost and bestPath
```

Listing 1: Least Cost Branch and Bound for TSP

7. Program

```cpp
#include <iostream>
#include <queue>
#include <vector>
#include <climits>
using namespace std;

int n;
int cost[10][10];
int bestCost = INT_MAX;
vector<int> bestPath;

struct Node {
    vector<int> path;
    int visited[10] = {0};
    int level;
    int currentCost;
    int bound;
};

struct Compare {
    bool operator()(Node a, Node b) {
        return a.bound > b.bound;
    }
};

int calculateBound(Node node) {
    int bound = node.currentCost;

    for (int i = 0; i < n; i++) {
        if (!node.visited[i] || i == node.path.back()) {
            int minCost = INT_MAX;

            for (int j = 0; j < n; j++) {
                if (i != j && cost[i][j] < minCost)
                    minCost = cost[i][j];
            }

            if (minCost != INT_MAX)
                bound += minCost;
        }
    }

```

```
    return bound;
}

void tsp() {
    priority_queue<Node, vector<Node>, Compare> pq;

    Node start;
    start.path.push_back(0);
    start.visited[0] = 1;
    start.level = 1;
    start.currentCost = 0;
    start.bound = calculateBound(start);

    pq.push(start);

    while (!pq.empty()) {
        Node curr = pq.top();
        pq.pop();

        if (curr.bound >= bestCost)
            continue;

        if (curr.level == n) {
            int total = curr.currentCost
                        + cost[curr.path.back()][0];

            if (total < bestCost) {
                bestCost = total;
                bestPath = curr.path;
                bestPath.push_back(0);
            }

            continue;
        }

        int last = curr.path.back();

        for (int i = 0; i < n; i++) {
            if (!curr.visited[i] && cost[last][i] > 0) {
                Node child = curr;
                child.path.push_back(i);
                child.visited[i] = 1;
                child.level = curr.level + 1;
                child.currentCost =
                    curr.currentCost + cost[last][i];
                child.bound = calculateBound(child);

                if (child.bound < bestCost)
                    pq.push(child);
            }
        }
    }
}

int main() {
    cout << "Enter number of cities (max 10): ";
    cin >> n;

    if (n < 2 || n > 10) {
        cout << "Invalid number of cities.\n";
```

```
        return 0;
    }

    cout << "Enter the cost matrix (0 means no direct road):\n";

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            cin >> cost[i][j];
        }
    }

    tsp();

    if (bestCost == INT_MAX) {
        cout << "No complete tour exists.\n";
    } else {
        cout << "\nMinimum Cost: " << bestCost;
        cout << "\nOptimal Route: ";

        for (int city : bestPath)
            cout << city << " ";

        cout << "\n";
    }

    return 0;
}
```

Listing 2: C++ Implementation of TSP using Least Cost Branch and Bound

8. Complexity Analysis

The Branch and Bound method reduces unnecessary exploration through lower-bound calculations and pruning. However, its worst-case performance remains factorial when pruning is ineffective.

Measure	**Complexity**	**Description**
Best-case search	Problem-dependent	Strong pruning can eliminate most branches, resulting in very few expanded nodes.
Worst-case time	$O(N!)$ search	If pruning is ineffective, the algorithm may explore factorially many possible routes.
Bound calculation	$O(N^2)$	The implementation scans the cost matrix to calculate the lower-bound estimate for a node.
Memory	Problem-dependent	The priority queue stores pending nodes and may grow large depending on pruning effectiveness.

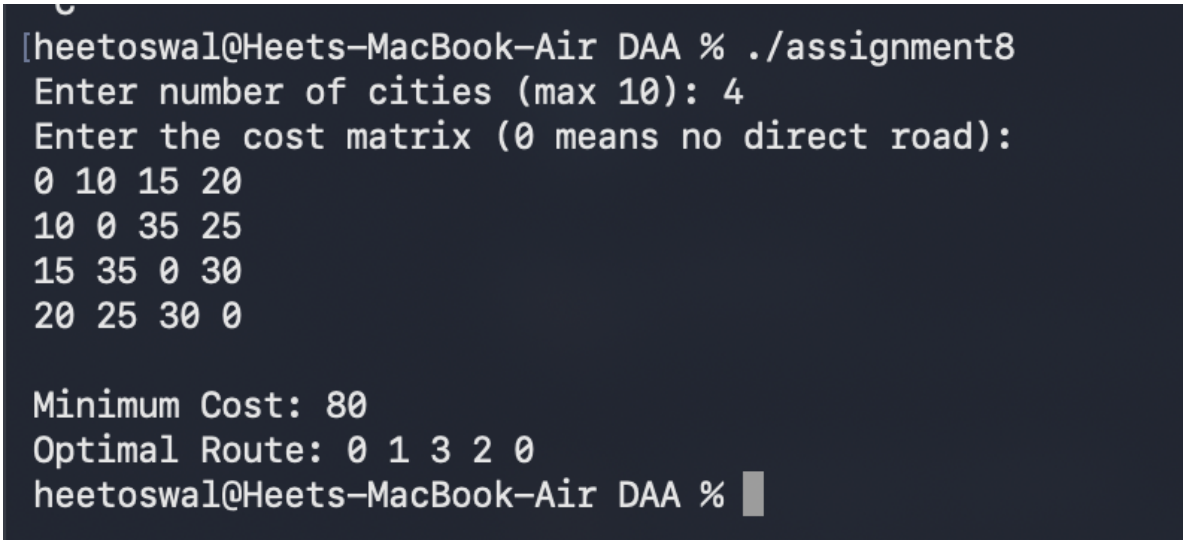
Here, N is the number of cities. In the submitted implementation, the cost matrix and visited array use $O(N^2)$ and $O(N)$ space respectively. The priority queue and stored paths can require substantially more memory during the search.

9. Notes and Limitations

- The implementation supports a maximum of 10 cities because the arrays are declared with fixed size 10.
- A matrix entry of zero is interpreted as the absence of a direct road when generating the next city.
- The algorithm starts from city 0 and returns to city 0 after visiting every city.
- The implementation does not explicitly model delivery time windows, fuel consumption changes, or room-like resource constraints.
- The lower-bound estimate in the submitted code is not the same as the reduced-cost matrix method presented in the reference manual.

10. Output

The output screenshot of the executed program is shown below.



11. Conclusion

The Travelling Salesman Problem is an optimization problem that requires finding a minimum-cost route visiting every city exactly once and returning to the starting point.

The submitted C++ implementation uses Least Cost Branch and Bound with a priority queue and a lower-bound estimate to prioritize promising partial routes and prune branches that cannot improve the current best solution.

Although the worst-case running time remains factorial, effective pruning can substantially reduce the search space. The approach is useful for moderate-sized route optimization problems where an exact solution is required.